

AI Agent Skills for Integration

If your team uses AI coding assistants like **Cursor**, **GitHub Copilot**, or **Windsurf**, you can provide them with specific "Agent Skills" (or rules) to help them integrate the Trimble Assist UI into your application flawlessly.

Instead of hoping the AI guesses the correct SDK methods, you can copy and paste the markdown files below into your project's `.cursor/rules` directory (for Cursor) or provide them as custom instructions. When you ask your AI assistant to perform a task, it will read these rules and write code that adheres to our SDK's best practices.

1. Core Integration Skill

This skill teaches the AI how to create the base React component that hosts the iframe, establish the connection, and manage the lifecycle to prevent memory leaks.

Suggested filename: `.cursor/rules/iframe-chat-integration.mdc`

```
---
description: Guides the agent on how to integrate the Trimble Assist
chat UI depending on the framework and its version.
globs: *.tsx, *.ts, *.jsx, *.vue, *.html
---

# Trimble Assist UI - Integration

When the user asks to add a chat interface, integrate Trimble Assist,
set up an agentic chat, or embed a chat iframe, follow these rules to
build a full-screen layout:

1. Framework Detection & Installation:
  - First, read the `package.json` to determine the framework and its
    version.
  - React:
    - If React version >= 18: run `npm install @trimble-agentic-
      external-npm-local/agentic-chat-react`
```

```

- If React version < 18: run `npm install @trimble-agentic-external-npm-local/agentic-platform-sdk-iframe-typescript@1.0.1-rc.7`
- **Vue:**
- If Vue version >= 3: run `npm install @trimble-agentic-external-npm-local/agentic-chat-vue`
- If Vue version < 3: run `npm install @trimble-agentic-external-npm-local/agentic-platform-sdk-iframe-typescript@1.0.1-rc.7`
- **Angular:**
- If Angular version >= 21: run `npm install @trimble-agentic-external-npm-local/agentic-chat-angular`
- If Angular version < 21: run `npm install @trimble-agentic-external-npm-local/agentic-platform-sdk-iframe-typescript@1.0.1-rc.7`
- **Other (None of the above):**
- run `npm install @trimble-agentic-external-npm-local/agentic-platform-sdk-iframe-typescript@1.0.1-rc.7`
- Ensure there is a valid `.npmrc` in the project root to resolve the Trimble custom registry.

```

2. ****Required Imports:****

```

- **React:** Import `AgenticMinimalChat` and `AgenticFullChat` from `@trimble-agentic-external-npm-local/agentic-chat-react`.
- **Angular:** Import `AgenticMinimalChat`, `AgenticFullChat`, and `AgenticNarrowChat` from `@trimble-agentic-external-npm-local/agentic-chat-angular`.
- **Vue:** Import `AgenticMinimalChat` and `AgenticFullChat` from `@trimble-agentic-external-npm-local/agentic-chat-vue`.
- **Base Iframe SDK:** Import `listenToChatUi`, `CHAT_UI_URLS`, `ChatUiConfiguration`, `ContentVariants`, and `ChatUiVariants` from `@trimble-agentic-external-npm-local/agentic-platform-sdk-iframe-typescript`.

```

3. ****State and Persistence:****

```

- Manage `variant` state. Allowed values depend on the framework:
  - **React / Vue:** `'minimal' | 'full'`.
  - **Angular:** `'minimal' | 'full' | 'narrow'` (the Angular wrapper additionally exposes `AgenticNarrowChat`).
- Manage `theme` (`'light' | 'dark') state.

```

4. ****Layout and Styling:****

```

- Ensure the chat component fully covers its provided wrapping area.
- Use a flex layout (`display: flex`, `flexDirection: 'column'`, `width: '100%'`, `height: '100%'`) where the chat wrapper expands to fill all available space (`flex: 1`, `overflow: 'hidden'`).

```

5. ****Component Rendering & Iframe Setup:****

```

- Conditionally render the chat component based on the `variant`

```

```
state (`AgenticMinimalChat`, `AgenticFullChat`, or, if it the Angular wrapper, `AgenticNarrowChat`).
```

- Create a token provider callback (e.g., wrapping ``getAccessTokenSilently()``).
- ****For Framework Wrappers (React, Angular, Vue):****
 - Pass the required props to the components. ****CRITICAL:** Note the specific prop interfaces for each framework:
 - ****React**:** ``provideToken`, `theme`, `agentId`, `environment`, `chatInputConfiguration``
 - ****Angular**:** ``[getAccessToken]`, `[theme]`, `[agentId]`, `[environment]`, `[chatInputConfiguration]``
 - ****Vue**:** ``:getAccessToken`, `theme` (or `:theme`), `:agentId`, `environment` (or `:environment`), `:chatInputConfiguration``
 - Ensure ``chatInputConfiguration`` is provided (e.g., ``{ buttons: [], hideModelSelection: false }``).
 - Set ``environment`` to `'stage'` or `'prod'`.
- ****For Base Iframe SDK:****
 - Render a standard `<iframe>` element with ``src={CHAT_UI_URLS[environment]}`` and full width/height.
 - Use a lifecycle hook (like ``useEffect`` or ``onMounted``) to initialize communication.
 - Define a ``getConfig`` function returning a ``ChatUiConfiguration`` object (containing ``environment`, `agentId`, `uiConfig`, etc.`).
 - Call ``listenToChatUi(iframeElement, iframeUrl, getConfig, getToken)`` and ensure you call the returned ``unsubscribe`` function on cleanup.

6. ****Credentials Handling:****

- Before generating the code, you ****must**** announce to the user that an ``accessToken`` and ``agentId`` are required.
- Ask the user to provide them. If they do not have them yet, ask if they want to proceed with placeholder values.
- If they choose to proceed with placeholders, leave a clear ``// TODO:`` comment exactly where these values are needed.
- Example (React Wrapper):


```
```typescript
// TODO: Please provide your accessToken and agentId here
const agentId = "YOUR_AGENT_ID";
const provideToken = async () => {
 return "YOUR_ACCESS_TOKEN";
};
```
```
- Example (Angular / Vue Wrappers):


```
```typescript
// TODO: Please provide your accessToken and agentId here
const agentId = "YOUR_AGENT_ID";
const getAccessToken = async () => {
```

```

 return "YOUR_ACCESS_TOKEN";
 };
 ...
- Example (Base Iframe SDK):
```typescript
// TODO: Please provide your accessToken and agentId here
const agentId = "YOUR_AGENT_ID";
const getToken = async () => {
    return "YOUR_ACCESS_TOKEN";
};
const getConfig = (): ChatUiConfiguration => ({
    environment: "stage",
    agentId,
    uiConfig: {
        theme: "light",
        contentVariant: ContentVariants.Chat,
        variant: ChatUiVariants.Full,
        chatInput: { buttons: [], hideModelSelection: false },
    },
    localization: { locale: "en" },
});
```

```

## 2. Authentication & Token Management Skill

This skill guides the AI on how to handle token provisioning and sign-in flows within the iframe, specifically interacting with Trimble ID.

**Suggested filename:** `.cursor/rules/iframe-chat-auth.mdc`

```

description: Guides the agent on how to handle token provisioning and
sign-in flows within the Trimble Assist iframe.
globs: *.tsx, *.ts, *.jsx

Trimble Assist UI - Authentication

When asked about "chat authentication", "chat token", "sign in prompt",
or "trimble id chat", follow these rules:

```

1. **\*\*Token Provisioning:\*\*** Pass a token provider function (e.g., ``getAccessTokenSilently`` from ``useAuth``) as the 4th argument to ``listenToChatUi``.
2. **\*\*Sign-In Modes:\*\***
  - `_Provide on startup:` Pass the token provider immediately.
  - `_Prompt sign-in:` Pass a function that resolves to an empty string ``() => Promise.resolve('')`` initially, and set ``showSignIn: true`` in the config.
3. **\*\*Event Listening:\*\***
  - Use ``listenToChatUiEvents`` to listen for ``ChatUiEventTypes.OnSignIn``.
  - When this event fires: update the token provider ref to the actual token function, and call ``updateConfig`` with ``showSignIn: false`` to hide the prompt.
  - Always clean up the event listener on component unmount.

## 3. Configuration & Customization Skill

This skill teaches the AI how to construct, type, and modify the `ChatUiConfiguration` object to customize the look and feel of the chat.

**Suggested filename:** `.cursor/rules/iframe-chat-configuration.mdc`

```

description: Guides the agent on how to construct, type, and modify the
ChatUiConfiguration object.
globs: *.tsx, *.ts, *.jsx

Trimble Assist UI - Configuration

When asked to "configure chat ui", "change chat variants", "update chat
theme", or "hide model selection", follow these rules:

1. **Type Definitions:** Import and use the `ChatUiConfiguration`,
`ChatUiVariants`, and `ContentVariants` types from the SDK.
2. **Configuration Structure:** Construct the configuration object
with:
 - `agentId`: The target agent ID.
 - `environment`: The target environment (e.g., `development`,
`production`).
 - `uiConfig`: The nested object containing visual settings.

```

```
3. **UI Customization:**
 - Set `variant` using `ChatUiVariants` (e.g.,
 `ChatUiVariants.Full`).
 - Set `contentVariant` using `ContentVariants` (e.g.,
 `ContentVariants.Chat`).
 - Configure the `chatInput` object (e.g., `hideModelSelection:
 false`, custom `buttons`).
```

## 4. Complete Minimal Example Skill

This skill provides the AI with a complete, working minimal React component that it can use as a starting template.

**Suggested filename:** `.cursor/rules/iframe-chat-minimal-example.mdc`

```

description: Provides a complete minimal React component template for
embedding the Trimble Assist iframe.
globs: *.tsx, *.jsx

```

```
Trimble Assist UI - Minimal Example
```

When asked to "create a minimal chat component" or "give me a working example of the chat iframe", use the following template.

```
Important Instructions for the Agent:
```

```
- Announce to the user that an `agentId` and an access token are
required.
- Request these values from the user. If they don't have them, ask if
they want to proceed with placeholder values.
- Keep the environment set to `stage` by default unless requested
otherwise.
```

```
```tsx
import { useEffect, useRef, useState } from "react";
import {
  CHAT_UI_URLS,
  listenToChatUi,
  type ChatUiConfiguration,
  ChatUiVariants,
```

```
    ContentVariants,
  } from "@trimble-agentic-external-npm-local/agentic-platform-sdk-
  iframe-typescript";

// TODO: Replace with your actual agent ID
const AGENT_ID = "YOUR_AGENT_ID";
// Default to stage environment
const ENVIRONMENT = "stage";

export const MinimalChat = () => {
  const iFrameRef = useRef<HTMLIFrameElement>(null);
  const cleanupRef = useRef<() => void> | null>(null);
  const [isRendered, setIsRendered] = useState(false);

  // TODO: Replace with your actual token retrieval logic
  const getAccessToken = async () => {
    return "YOUR_ACCESS_TOKEN";
  };

  const getConfig = (): ChatUiConfiguration => ({
    agentId: AGENT_ID,
    environment: ENVIRONMENT,
    uiConfig: {
      theme: "light",
      contentVariant: ContentVariants.Chat,
      variant: ChatUiVariants.Full,
      chatInput: { buttons: [], hideModelSelection: false },
    },
    localization: { locale: "en" },
  });

  useEffect(() => {
    if (!isRendered || !iFrameRef.current) return;

    cleanupRef.current = listenToChatUi(
      iFrameRef.current,
      CHAT_UI_URLS[ENVIRONMENT],
      getConfig,
      getAccessToken,
    );

    return () => {
      cleanupRef.current?.();
    };
  }, [isRendered]);

  return (
```

```
<div style={{ height: "100vh", width: "100%" }}>
  {!isRendered ? (
    <button onClick={() => setIsRendered(true)}>Start Chat</button>
  ) : (
    <iframe
      ref={iFrameRef}
      src={CHAT_UI_URLS[ENVIRONMENT]}
      style={{ width: "100%", height: "100%", border: "none" }}
    />
  )}
</div>

);

};

`;
```